

فصل ۸

بن‌بست

در یک محیط چندبرنامگی امکان دارد که چندین نخ برای دستیابی به تعدادی منبع متناهی به رقابت بپردازند. هر نخ منابعی درخواست می‌کند؛ اگر منابع در آن لحظه در دسترس نباشند، نخ وارد یک حالت انتظار می‌شود. گاهی اوقات، نخی که منتظر است هرگز نمی‌تواند دوباره تغییر حالت بدهد، چون منابعی که درخواست کرده است در اختیار نخ‌های منتظر دیگری است. این وضعیت **بن‌بست** نامیده می‌شود. این موضوع را در فصل ۶ به‌اختصار با شکلی از شکست عاقبت به‌خیری بحث کردیم. ما در آنجا، بن‌بست را به‌صورت یک وضعیتی تعریف کردیم که در آن، *هر پردازه در مجموعه‌ای از پردازه‌ها به‌انتظار رویدادی است که می‌تواند تنها توسط پردازه‌ی دیگری در همان مجموعه به‌وقوع بپیوندد.*

شاید بهترین نمود بن‌بست را بتوان از قانونی که در اوایل قرن بیستم به تصویب مجلس تگزاس رسید، استنباط کرد. در بخشی از این قانون آمده است: «زمانی که دو قطار در تقاطعی به هم می‌رسند، بایستی هر دو به طور کامل متوقف شوند و هیچ یک از آنها دوباره حرکت نمی‌کند، مگر این که دیگری رفته باشد.»

در این فصل، روش‌هایی را مطرح می‌کنیم که برنامه‌نویسان کاربردی در کنار برنامه‌نویسان سیستم‌عامل می‌توانند برای پیشگیری یا مواجهه با بن‌بست‌ها به‌کار ببرند. اگرچه برخی برنامه‌های کاربردی می‌توانند برنامه‌هایی را که در بن‌بست هستند شناسایی کنند، سیستم‌عامل‌ها نوعاً امکاناتی برای پیشگیری از بن‌بست‌ها فراهم نمی‌کنند و مسؤولیت برنامه‌نویسان است که اطمینان بدهند برنامه‌های عاری از بن‌بست را طراحی می‌کنند. مسائل بن‌بست—همین‌طور، سایر شکست‌های عاقبت به‌خیری—همچنان‌که تقاضا برای ازدیاد همروندی و موازی‌سازی بر روی سیستم‌های چندهسته‌ای تداوم پیدا می‌کند، چالش بیشتری را مطرح می‌کنند.

اهداف فصل

- نمایش چگونگی وقوع بن‌بست موقعی که قفل‌های انحصار متقابل استفاده می‌شوند.
- تعریف چهار شرط لازمی که مشخصه‌ی بن‌بست هستند.
- مشخص‌کردن یک وضعیت بن‌بست در یک گراف تخصیص منبع.
- ارزیابی چهار رویکرد متفاوت برای پیشگیری بن‌بست‌ها.
- به کارگیری الگوریتم بانکدار برای اجتناب از بن‌بست.

- به کارگیری الگوریتم تشخیص بن‌بست.
- ارزیابی رویکردهایی برای احیا از بن‌بست.

۸-۱ مدل سیستم

سیستم شامل تعدادی منبع متناهی است که مابین تعدادی نخ رقابت‌کننده توزیع می‌شوند. این منابع ممکن است به چندین نوع (یا رده) که هر یک شامل تعدادی نمونه‌ی یکسان هستند، تقسیم شوند. دورهای پردازنده، فایل‌ها و دستگاه‌های I/O (از قبیل رابط‌های شبکه و DVD گردان‌ها)، مثال‌هایی از انواع منابع به شمار می‌روند. اگر سیستمی چهار پردازنده داشته باشد، در این صورت نوع منبع پردازنده چهار نمونه دارد. به‌طور مشابه، نوع منبع شبکه می‌تواند دو نمونه داشته باشد. اگر نخی یک نمونه از یک نوع منبع را درخواست نماید، این درخواست بایستی با تخصیص **هر یک** از نمونه‌های نوع منبع موجود قابل ارضا باشد. اگر غیر از این باشد، در این صورت نمونه‌ها یکسان نبوده و رده‌های نوع منابع به درستی تعریف نشده‌اند.

ابزار مختلف همگامی بحث شده در فصل ۶، همچون قفل‌های انحصار متقابل و سمافورها نیز منابع سیستمی هستند؛ و در سیستم‌های کامپیوتری فعلی، آن‌ها رایج‌ترین سرچشمه‌های بن‌بست محسوب می‌شوند. با این وجود، اینجا تعریف مسئله نیست. یک قفل نوعاً با یک ساختمان‌داده‌ی خاصی مرتبط است–یعنی، یک قفل ممکن است برای حفاظت دسترسی یک صف، دیگری برای حفاظت دسترسی به یک لیست پیوندی و از این دست استفاده شوند. به این دلیل، هر نمونه از یک قفل نوعاً برای رده‌ی منبع خاص خود گمارده می‌شود.

توجه داشته باشید که ما در طول این فصل، منابع هسته را مورد بحث قرار می‌دهیم، اما نخ‌ها ممکن است منابعی را هم از پردازه‌های دیگر استفاده کنند (برای مثال، از طریق ارتباط بین پردازه‌ای) و این استفاده‌های منابع نیز می‌تواند به بن‌بست منجر شود. این قبیل بن‌بست‌ها دغدغه‌ی هسته نبوده و لذا در اینجا مطرح نمی‌شوند.

هر نخ بایستی پیش از استفاده‌ی منبع، آن را درخواست نموده و بعد از استفاده رها سازد. هر نخ برای انجام کار محوله‌اش می‌تواند به هر تعداد از منابع که نیاز دارد، درخواست نماید. بدیهی است که تعداد منابع درخواستی نمی‌تواند از تعداد کل منابع موجود سیستم فراتر رود. به عبارت دیگر، یک نخ نمی‌تواند در سیستمی که تنها یک رابط شبکه دارد، درخواستی برای سه رابط انجام بدهد.

یک نخ در شرایط معمول عملیات سیستم فقط با پیروی از دنباله‌ی زیر می‌تواند از منبعی بهره‌مند شود:

۱- **درخواست**: نخ منبع را درخواست می‌نماید. اگر درخواست نتواند بلافاصله اعطا شود (برای مثال، در صورتی‌که یک قفل انحصار متقابل، فعلاً توسط نخ دیگری نگه داشته شده است)، در این صورت نخی درخواست‌کننده بایستی منتظر شود تا این که بتواند منبع را کسب کند.

۲- **استفاده**: نخ می‌تواند روی منبع عملیات داشته باشد (برای مثال، اگر منبع یک قفل انحصار متقابل باشد، نخ می‌تواند بخش بحرانی خود را دسترسی کند).

۳- **رهاسازی**: نخ منبع را رها می‌سازد.

درخواست و رهاسازی منابع، چنان که در فصل ۲ تشریح شد، ممکن است فراخوانی‌های سیستمی باشند. فراخوانی‌های سیستمی request() و release() یک دستگاه، open() و close() یک فایل و allocate() و free() حافظه، مثال‌هایی از این دست هستند. به طور مشابه، همان‌گونه که در فصل ۶ دیدیم، درخواست و رهایی می‌تواند از طریق عملیات wait() و signal() روی سمافورها و از طریق کسب و رهایی یک قفل انحصار متقابل انجام شود. در رابطه با هر یک از منابع تحت مدیریت هسته که هر نخ استفاده می‌کند، سیستم‌عامل جهت اطمینان بررسی می‌کند که منبع درخواستی نخ تخصیص می‌یابد یا خیر. یک جدول سیستمی، تخصیص و یا آزاد بودن هر منبع را ثبت می‌کند. هر منبعی که تخصیص می‌یابد، نخی که منبع به آن تخصیص می‌یابد نیز در جدول ثبت می‌شود. اگر یک نخ، منبعی را درخواست نماید که فعلاً به نخ دیگری تخصیص یافته است، می‌تواند به صف نخ‌های منتظر آن منبع افزوده شود.

یک مجموعه از نخ‌ها در حالت بن‌بست هستند، هرگاه هر یک از نخ‌های آن مجموعه در انتظار رویدادی باشند که تنها توسط یکی از نخ‌های همان مجموعه می‌تواند اتفاق بیفتد. رویدادهایی که بیشتر در اینجا مد نظر ما هستند کسب و رهایی منبع می‌باشند. منابع نوعاً منطقی (برای مثال، قفل‌های انحصار متقابل، سمافورها، و فایل‌ها) هستند؛ با این وجود، انواع دیگری از رویدادها نیز ممکن است منجر به وقوع بن‌بست‌ها شوند، اعم از خواندن از یک رابط شبکه یا امکانات IPC (ارتباط بین پردازه‌ای) که در فصل ۳ بحث شدند.

ما برای روشن‌شدن حالت بن‌بست به مسئله‌ی فیلسوفان خوش‌خوراک از بخش ۷-۱-۳ بر می‌گردیم. در این شرایط، منابع با چوبک‌ها نمایش داده می‌شوند. اگر تمامی فیلسوفان به‌طور همزمان گرسنه شوند، و هر فیلسوف چوبک سمت چپ خود را بردارد، دیگر هیچ چوبک دیگری باقی نمی‌ماند. در ادامه، هر فیلسوف به‌انتظار مهیاشدن چوبک سمت راست خودش مسدود می‌شود.

برنامه‌نویسان برنامه‌های چندنخی بایستی از احتمال وقوع بن‌بست‌ها مطلع باشند. ابزار قفل‌گذاری مطرح شده در فصل ۶ برای اجتناب از شرایط مسابقه طراحی می‌شوند. با این وجود، در استفاده از این ابزار، برنامه‌نویسان بایستی در چگونگی کسب و رهایی قفل‌ها دقت کافی مبذول نمایند. در غیر این صورت، بن‌بست می‌تواند اتفاق بیفتد، چنان‌چه در ادامه بیان می‌شود.

۸-۲ بن‌بست در برنامه‌های چندنخی

پیش از بررسی چگونگی شناسایی و مدیریت مسئله‌های بن‌بست، در ابتدا چگونگی وقوع بن‌بست در یک برنامه‌ی چندنخی که از قفل‌های انحصار متقابل پوزیکس استفاده می‌کند را نشان می‌دهیم. تابع pthread_mutex_init() یک قفل انحصار متقابل قفل نشده را مقدار اولیه می‌دهد. قفل‌های انحصار متقابل به ترتیب با استفاده از توابع pthread_mutex_lock() و pthread_mutex_unlock() کسب و رها می‌گردند. اگر یک نخ در صدد کسب یک قفل انحصار متقابل قفل‌شده‌ای باشد، فراخوانی pthread_mutex_lock() نخ را مسدود می‌سازد تا این که مالک قفل انحصار متقابل، تابع pthread_mutex_unlock() را احضار نماید.

در نمونه کد زیر، دو قفل انحصار متقابل ایجاد شده و مقدار اولیه می‌گیرند:

```
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;

pthread_mutex_init(&first_mutex,NULL);
pthread_mutex_init(&second_mutex,NULL);
```

در ادامه، دو نخ—thread_one و thread_two—ایجاد می‌شوند و هر دوی این نخ‌ها به هر دوی قفل‌های انحصار متقابل دسترسی دارند. نخ‌های thread_one و thread_two، به ترتیب در توابع do_work_one() و do_work_two()، چنان‌چه در شکل ۸-۱ نشان داده می‌شوند، اجرا دارند.

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
        pthread_mutex_lock(&first_mutex);
        pthread_mutex_lock(&second_mutex);
        /**
         * Do some work
         */
        pthread_mutex_unlock(&second_mutex);
        pthread_mutex_unlock(&first_mutex);
        pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
        pthread_mutex_lock(&second_mutex);
        pthread_mutex_lock(&first_mutex);
        /**
         * Do some work
         */
        pthread_mutex_unlock(&first_mutex);
        pthread_mutex_unlock(&second_mutex);
        pthread_exit(0);
}
```

شکل ۸-۱ مثال بن‌بست.

در این مثال، thread_one قصد دارد قفل‌های انحصار متقابل را به ترتیب (۱) first_mutex، (۲) second_mutex به‌دست آورد. همزمان، thread_two در صدد است تا قفل‌های انحصار متقابل را به

ترتیب (۱) second_mutex، (۲) first_mutex کسب نماید. اگر thread_one قفل first_mutex و thread_two قفل second_mutex را به‌دست آورند، بن‌بست اتفاق می‌افتد.

توجه داشته باشید که هرچند احتمال وقوع بن‌بست وجود دارد، اما اگر thread_one بتواند پیش از آنکه thread_two قفلی کسب کند هر دو قفل first_mutex و second_mutex را در اختیار بگیرد، بن‌بست به وجود نخواهد آمد. و البته، ترتیبی که نخ‌ها با آن اجرا می‌شوند به چگونگی زمانبندی آنها توسط پردازنده بستگی دارد. این مثال، مسأله‌ای را در رابطه با مواجهه با بن‌بست‌ها نشان می‌دهد: شناسایی و آزمون بن‌بست‌هایی که تحت شرایط زمان‌بندی خاصی به وجود می‌آیند مشکل است.

۸-۲-۱ قفل زنده

قفل زنده[1] شکل دیگری از شکست عاقبت به‌خیری است. آن شبیه به بن‌بست است؛ هر دو از پیشروی دو یا چند نخ جلوگیری می‌کنند، اما نخ‌ها به‌دلایل متفاوتی قادر به پیشروی نیستند. در حالی که بن‌بست زمانی اتفاق می‌افتد که هر نخ در یک مجموعه به‌انتظار یک رویدادی که می‌تواند تنها توسط نخ دیگری از همان مجموعه اتفاق بیفتد مسدود می‌شود، قفل زنده موقعی اتفاق می‌افتد که یک نخ به‌صورت مداوم تلاش می‌کند تا عملی انجام بدهد که با شکست مواجه می‌شود. قفل زنده شبیه به چیزی است که اتفاق می‌افتد، وقتی دو نفر تلاش می‌کنند تا از یک سالن عبور کنند: یکی به سمت راستش حرکت می‌کند، دیگری به سمت چپش که این کار مانع پیشروی دیگری می‌شود. در ادامه، یکی به سمت چپش حرکت می‌کند و دیگری به سمت راستش و الی‌آخر. آن‌ها مسدود می‌شوند، اما هیچ‌گونه پیشروی ندارند.

قفل زنده می‌تواند با تابع pthrread_mutex_trylock() نمایش داده شود که تلاش می‌کند تا یک قفل انحصار متقابل را بدون انسداد کسب کند. کد نمونه‌ی شکل ۸-۲ مثال شکل ۸-۱ را بازنویسی می‌کند طوری که اکنون از pthread_mutex_trylock() استفاده می‌کند. این وضعیت می‌تواند به قفل زنده منجر شود، در صورتی که thread_one انحصار متقابل first_mutex را کسب کرده و به‌دنبال آن، thread_two انحصار متقابل second_mutex را کسب کند. در ادامه، هر کدام از نخ‌ها pthread_mutex_trylock() را احضار می‌کنند که با شکست مواجه می‌شوند، قفل‌های در اختیارشان را رها می‌کنند، و همان اَعمال را به‌صورت نامتناهی تکرار می‌کنند.

قفل زنده نوعاً زمانی اتفاق می‌افتد که نخ‌ها عملیات منجر به شکستی را به‌صورت همزمان بارها و بارها تکرار می‌کنند. لذا عموماً می‌تواند با اجبار هر نخ به انجام مجدد عملیات شکست در زمان‌های تصادفی اجتناب شود. این کار دقیقاً رویکردی است که توسط شبکه‌های اترنت دنبال می‌شود موقعی که یک برخورد شبکه پیش می‌آید. به‌جای این که بعد از وقوع یک برخورد، تلاش شود دومرتبه یک بسته ارسال شود، میزبان مشمول برخورد، قبل از اقدام مجدد برای ارسال، یک دوره‌ی زمانی تصادفی **عقب‌نشینی**[2] خواهد کرد.

```
/* thread one runs in this function */
```

1. Livelock
2. Backoff

```
void *do_work_one(void *param)
{
        int done = 0;
        while (!done) {
                pthread_mutex_lock(&first_mutex);
                if (pthread_mutex_trylock(&second_mutex)) {
                        /**
                        * Do some work
                        */
                        pthread_mutex_unlock(&second_mutex);
                        pthread_mutex_unlock(&first_mutex);
                        done = 1;
                }
                else
                        pthread_mutex_unlock(&first_mutex);
        }
        pthread_exit(0);
}

/* thread two runs in this function */
void *do_work_two(void *param)
{
        int done = 0;
        while (!done) {
                pthread_mutex_lock(&second_mutex);
                if (pthread_mutex_trylock(&first_mutex)) {
                        /**
                        * Do some work
                        */
                        pthread_mutex_unlock(&first_mutex);
                        pthread_mutex_unlock(&second_mutex);
                        done = 1;
                } else
                        pthread mutex unlock(&second mutex);
        }
        pthread exit(0);
}
```

شکل ۸–۲ مثال قفل زنده.

قفل زنده نسبت به بن‌بست رواج کمتری دارد اما با این همه یک موضوع چالش‌برانگیز در طراحی برنامه‌های همروند می‌باشد و مثل بن‌بست فقط ممکن است تحت شرایط زمان‌بندی خاصی پیش بیاید.